TP Algorithmique n°2

Les structures conditionnelles

Objectifs de la séance.
Faire prendre des décisions à un programme (`if` / `else`, puis `elif`) : d'abord **lire et prévoir**, puis **modifier** et corriger des fonctions, enfin **créer** les siennes ; toujours tester les **valeurs frontières**.

Barème. TP noté sur 20 (points indiqués à chaque exercice) ; le quiz final est noté à part, sur 5.

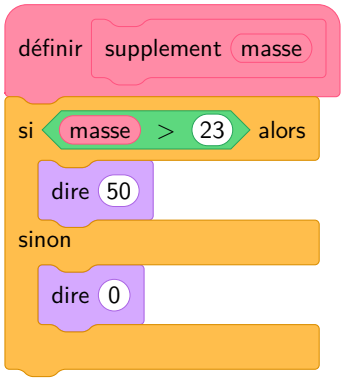
Se connecter à l'espace élève, onglet **TP Python**, puis saisir le code d'accès donné au tableau (majuscules et tiret compris). Ce document est la trace écrite : on y note ses **prévisions avant** d'exécuter.

Partie 1 — Cours

L'instruction conditionnelle « si… sinon… »

Une compagnie aérienne fait payer un **supplément de** 50 € lorsqu'une valise pèse **plus de** 23 **kg** ; sinon, il n'y a rien à payer.

En Scratch



En Python

Script

```
1 def supplement(masse):
2     if masse > 23:
3         return 50
4     else:
5         return 0
```

À retenir :

1. Si la condition est vraie, Python exécute le bloc sous `if` ; sinon, le bloc sous `else`.
2. Deux-points : après la condition et après `else`, puis on décale les lignes (indentation).

En maths	si $x < 3$	si $x > 3$	si $x \leqslant 3$	si $x \geqslant 3$	si $x = 3$	si $x \neq 3$
En Python	`if x < 3:`	`if x > 3:`	`if x <= 3:`	`if x >= 3:`	`if x == 3:`	`if x != 3:`

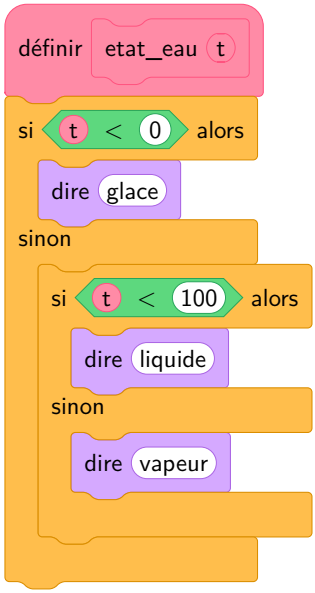
Pour **tester** une égalité, on écrit `==` ; le simple `=` est réservé à l'affectation.

Point vocabulaire

Le **nom** de la fonction est `supplement` ; elle possède **un seul paramètre**, `masse`. Dans la console, on **appelle** la fonction en donnant une valeur au paramètre, par exemple `supplement(25)` : elle **renvoie** un résultat.

★ Nouveau : plus de deux cas avec « sinon si » (`elif`)

En Scratch (deux « si… sinon » emboîtés)



En Python

Script

```
def etat_eau(t):
    if t < 0:
        return "glace"
    elif t < 100:
        return "liquide"
    else:
        return "vapeur"
```

À retenir :

1. `elif` est la contraction de « *else if* » (sinon si).
2. Python teste les conditions **dans l'ordre**, de haut en bas, et exécute **seulement le premier bloc** dont la condition est vraie.
3. Le bloc `else` n'est exécuté que si aucune condition n'est vraie.
4. Par exemple, pour $t = 20$: $20 < 0$ est faux, puis $20 < 100$ est vrai, donc l'appel `etat_eau(20)` renvoie `"liquide"`.

Partie A — Lire et prévoir

Méthode. Pour chaque appel ou script, écrire d'abord sa **prévision**, sans ordinateur. Ensuite seulement, exécuter pour vérifier, et se corriger si besoin.

Exercice 1 ★☆☆ 1,5 pt — Raisonner

Si… sinon. On reprend la fonction `supplement` du cours.

Appel	Je prévois	Python renvoie
`supplement(25)`	.....	.....
`supplement(23)`	.....	.....
`supplement(23.5)`	.....	.....

Pourquoi ? `masse` vaut 23 : la condition `masse > 23` est ....., car ..... ; donc le bloc ..... est exécuté.

À retenir : une valise de 23 kg ne paie pas de supplément : attention aux **valeurs frontières**.

Exercice 2 ★☆☆ 1,5 pt — Raisonner

Plus de deux cas. On reprend la fonction `etat_eau` du cours.

Appel	Je prévois	Python renvoie
`etat_eau(-5)`	.....	.....
`etat_eau(0)`	.....	.....
`etat_eau(100)`	.....	.....

Pourquoi ? `t` vaut 100 : la condition `t < 0` est ..... ; on teste la condition suivante, `t < 100`, qui est ....., car ..... ; donc le bloc ..... est exécuté.

Exercice 3 ★★☆ 1,5 pt — Raisonner

Deux `if` à la suite.

Script C

```
1 a = 8
2 if a > 4:
3     a = 2 * a - 5
4 if a > 10:
5     a = a - 10
6 print(a)
```

Je prévois que le script affiche ..... .

Python affiche ..... .

Pourquoi ? `a` vaut 8 : la condition `a > 4` est ....., donc `a` prend la valeur ..... .

Le second `if` teste la **nouvelle** valeur de `a` : `a > 10` est ....., donc `a` prend la valeur ..... .

À retenir : deux `if` à la suite sont **indépendants** : ils sont testés l'un après l'autre. Avec `elif`, au contraire, un seul bloc est exécuté.

Partie B — Modifier et corriger

Exercice 4 ★☆☆ 1,5 pt — Raisonner

Corriger le script B.

Script B

```
1 note = 10
2 if note > 10:
3     print("admis")
4 else:
5     print("refusé")
```

Je prévois que le script affiche ..... .

Python affiche ..... .

Un élève qui a exactement 10 devrait être **admis**. Corriger la condition (sans changer la note) :

`if` ..... `:`

Exercice 5 ★★☆ 2 pt — Modéliser

Au cinéma. Un cinéma propose un tarif réduit de 6 € aux personnes de **moins de 18 ans** ; les autres paient 9 €.

Script

```
1 def tarif(age):
2     if age < 18:
3         return 6
4     else:
5         return 9
```

① Le cinéma crée un **tarif enfant** de 4,50 € pour les **moins de 14 ans**. Modifier la fonction `tarif` à l'aide de `elif` (4,50 s'écrit `4.5` en Python), puis compléter :

`tarif(10)` : ..... `tarif(14)` : ..... `tarif(17)` : ..... `tarif(18)` : .....

② Un élève a écrit les deux tests dans l'autre ordre :

Script

```
def tarif(age):
    if age < 18:
        return 6
    elif age < 14:
        return 4.5
    else:
        return 9
```

Je prévois que `tarif(10)` renvoie .....

Python renvoie .....

Pourquoi le cas `age < 14` n'est-il jamais atteint ?

Exercice 6 ★★☆ 2 pt

Chercher

Trouver le bug. Une école de rugby répartit ses joueurs : **U8** de 8 à 10 ans exclus, **U10** de 10 à 12 ans exclus, **U12** de 12 à 14 ans exclus ; avant 8 ans « Trop jeune », à partir de 14 ans « Hors école ». La fonction ci-dessous contient **deux erreurs**.

Script

```
def groupe(age):
    if age < 8:
        return "Trop jeune"
    elif age <= 10:
        return "U8"
    elif age < 14:
        return "U12"
    elif age < 12:
        return "U10"
    else:
        return "Hors école"
```

① Tester la fonction aux âges **frontières** et compléter.

age	7	8	10	11	12	13	14
Groupe attendu	.....	.....	.....	.....	.....	.....	.....
`groupe(age)` renvoie	.....	.....	.....	.....	.....	.....	.....

La fonction se trompe pour les âges .....

② Corriger les deux erreurs, puis tester de nouveau aux âges frontières.

Partie C — Créer

Exercice 7 ★★☆ 2 pt

Chercher

Le plus grand des deux. Écrire le corps de la fonction `maximum(a, b)`, qui renvoie le plus grand des deux nombres `a` et `b`, à l'aide de `if` et `else`. La tester avec `maximum(5, 5)` et `maximum(-2, -5)`.

```
def maximum(a, b):
```

. . . . . .

. . . . . .

. . . . . .

. . . . . .

`maximum(5, 5)` renvoie . . . . . `maximum(-2, -5)` renvoie . . . . . .

Exercice 8 ★★★ 3 pt

Modéliser

Page blanche : la mention au brevet. Au brevet, la mention dépend de la note sur 20 :

⋆ à partir de 16 : « Très bien » ; à partir de 14 : « Bien » ; à partir de 12 : « Assez bien » ;

⋆ à partir de 10 : « Passable » ; en dessous de 10 : « Insuffisant ».

① **Sans modèle**, écrire la fonction `mention(note)` qui renvoie la mention (texte entre guillemets).

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

. . . . . .

② La tester aux notes frontières et compléter.

`note`	9.5	10	13	14	16
`mention(note)`	. . . . .	. . . . .	. . . . .	. . . . .	. . . . .

Quiz final (noté sur 5)

Cinq questions rapides, **sans ordinateur** : cocher la bonne réponse.

1. Que renvoie l'appel `f(5)` ?

Script

```
def f(x):
    if x > 5:
        return "A"
    else:
        return "B"
```

❑ `"A"` ❑ `"B"` ❑ une erreur ❑ rien

4. Que renvoie l'appel `g(10)` ?

Script

```
def g(a):
    if a < 10:
        return 1
    elif a < 20:
        return 2
    else:
        return 3
```

❑ 1 ❑ 2 ❑ 3 ❑ une erreur

2. Pour tester si une personne a **au moins** 18 ans, on écrit :

❑ `if age > 18:` ❑ `if age >= 18:`
❑ `if age => 18:` ❑ `if age < 18:`

3. Dans une suite `if ... elif ... else`, combien de blocs sont exécutés ?

❑ un seul ❑ deux ❑ aucun
❑ tous ceux dont la condition est vraie

5. Qu'affiche ce script ?

Script

```
x = 4
if x > 3:
    x = x + 1
if x > 4:
    x = x * 2
print(x)
```

❑ 4 ❑ 5 ❑ 8 ❑ 10

Partie D — Défis

Exercice 9 ★★★ 2,5 pt

Modéliser

Frais de port. Un site de vente calcule les frais de port selon la masse m du colis (en kg) :

- ⋆ jusqu'à 1 kg : 5 € ;
- ⋆ de plus de 1 kg à 5 kg : 5 €, plus 2 € par kg au-dessus de 1 kg ;
- ⋆ au-delà de 5 kg : 13 €, plus 1 € par kg au-dessus de 5 kg.

① Calculer à la main : `port(3)` vaut ..... et `port(8)` vaut ..... .

② Écrire la fonction `port(m)`, puis vérifier avec la console les deux valeurs précédentes.

Exercice 10 ★★★ 2,5 pt

Raisonner

Les antécédents. On considère la fonction `f` suivante (automatisme de la Série 4) :

Script

```
def f(x):
    if x > 3:
        return 2 * x
    else:
        return x + 10
```

① Trouver **tous** les nombres x tels que `f(x)` renvoie 14. Étudier les deux cas ($x > 3$ et $x \leqslant 3$), puis vérifier avec la console.

② Même question avec 7.